



PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu¹ and Justin B. Kinney^{1,*}

¹Simons Center for Quantitative Biology, Cold Spring Harbor Laboratory, 1 Bungtown Rd., 11724, New York, United States

*Corresponding author. jkinney@cshl.edu

FOR PUBLISHER ONLY Received on Date Month Year; revised on Date Month Year; accepted on Date Month Year

Abstract

Summary: Computationally designed DNA sequence libraries (also called oligonucleotide pools) are essential components of massively parallel reporter assays (MPRAs), protein deep mutational scanning (DMS) experiments, and other multiplexed assays of variant effect (MAVEs). Increasingly, such libraries are also being used to analyze genomic AI models in silico. Despite their importance, the process of designing these libraries remains tedious and error-prone due to the lack of purpose-built software. Here we introduce PoolParty, a Python package that enables the streamlined design of sequence libraries using a declarative, composable syntax. In PoolParty, users build libraries by chaining together a core set of operations into a directed acyclic graph (DAG) that generates sequences on demand. This makes it possible to specify complex library designs (e.g., mixing multiple types of systematic and random variation across multiple sequence regions) in just a few lines of code. PoolParty automatically generates informative names for each sequence, as well as “design cards” that report the specific operations used to generate each sequence. Over 20 built-in operation types cover nucleotide- and codon-level mutagenesis, motif insertion, deletion and insertion scans, barcode generation, and more. Visualization methods let users quickly audit library content and inspect the underlying DAG. PoolParty thus transforms oligo pool design from a tedious task requiring custom functions and scripts into a structured, transparent, and reproducible process.

Availability and implementation: PoolParty can be installed using the pip package manager and is compatible with Python ≥ 3.10 . Documentation is provided at <http://poolparty.readthedocs.io>; source code is available at <http://github.com/jbkinney/poolparty>.

Contact: jkinney@cshl.edu (J.B.K.).

Introduction

Oligonucleotide pools have become essential tools in functional genomics. In massively parallel reporter assays (MPRAs), designed libraries of DNA sequences are routinely used to measure the regulatory activities of tens of thousands of candidate or variant regulatory elements in a single experiment [Inoue and Ahituv, 2015]. In deep mutational scanning (DMS) experiments, coding sequences that have been either systematically or randomly mutagenized are commonly used to map protein fitness landscapes [Fowler and Fields, 2014]. Both MPRAs and DMS experiments are examples of multiplexed assays of variant effect (MAVEs), a large and expanding family of high-throughput methods that quantitatively link sequence to function at scale [Kinney and McCandlish, 2019]. The same idea extends beyond the wet lab: complex DNA sequence libraries are increasingly being used in

silico to probe the behavior of genomic AI models [Avsec et al., 2021, Nagai et al., 2026].

Computationally designing these libraries is straightforward in principle, but in practice it is tedious and error-prone. Typical applications require oligo pools built from multiple types of variant sequences generated in multiple ways. A DMS library, for example, might combine exhaustive single-amino-acid substitutions, sampled higher-order mutants, wild-type replicates, and barcodes—each generated by a different procedure and subject to different coverage requirements. An MPRA library might tile multiple transcription factor binding sites across a regulatory element in all possible permutations and orientations, then assign each variant multiple barcodes. Coordinating this kind of combinatorial logic in a one-off script quickly becomes unwieldy, and often produces errors that are hard to catch.

Existing tools can help with parts of this process, but none provide a unified framework for library design. General-purpose

sequence toolkits [Cock et al., 2009, Schreiber, 2025, Pereira et al., 2015, Klie and Laub, 2023] can manipulate individual sequences but have no notion of a variant library as a structured object. Consequently, users must write custom scripts to handle the combinatorial logic themselves. Assay-specific tools automate library design for particular experimental formats but are not easily adapted to novel designs [Georgakopoulos-Soares et al., 2016, Ghazi et al., 2018, Barbon et al., 2021]. And sequence optimization tools such as DnaChisel [Zulkower and Rosser, 2020] and CodonGenie [Swainston et al., 2017] address synthesis constraints on individual sequences, but not library-level design.

Here we present PoolParty, a Python package that enables the streamlined design of complex DNA sequence libraries. In PoolParty, libraries are represented as objects called Pools, while individual steps in the sequence generation process are called Operations. Using a concise and transparent syntax, users chain together Operations into a directed acyclic graph (DAG) that yields the desired Pool. The DAG describes the high-level logic used to generate sequences, while the procedural logic and bookkeeping are handled internally. Importantly, no sequences are generated until the user requests them, and only the requested sequences are generated. This makes it possible for users to explore and test multiple design options without triggering expensive computations. When sequences are generated, each is automatically assigned an informative name and paired with a *design card* that records the specific procedures used to construct it. We demonstrate PoolParty in three example applications: a DMS library for protein GB1, an MPRA library probing transcription factor binding site grammar, and an *in silico* experiment characterizing the behavior of SpliceAI [Jaganathan et al., 2019] in which the design cards form the basis for surrogate modeling.

Methods

Pools and Operations

PoolParty is built around two core concepts: Pools and Operations. Pools represent collections of sequences—either the final library that the user wishes to generate, or intermediate sets of sequences out of which the final library is constructed. Operations represent the steps used to create Pools. When designing a library, users chain together multiple Operations into a DAG that yields the desired Pool. Sequences are then generated on demand from the final resulting Pool (the *root Pool*) using the DAG. By delaying sequence generation until it is explicitly requested, users can test many different library designs (inspecting a few sampled sequences from each) before committing to generating the full library.

PoolParty provides over 20 built-in Operations that users can choose from when constructing DAGs. Each Operation takes zero or more Pools as input and returns exactly one Pool as output. It is useful to think of Operations as falling into four functional categories (Fig. 1A): source, transformation, composition, and state Operations. *Source Operations* create the initial Pools from which all downstream Pools are constructed. These may represent user-specified sequences, DNA sequences generated using position weight matrices or IUPAC motifs, random k-mers, barcodes, and so on. *Transformation Operations* modify sequence content, e.g., generating sequence variants through point mutations, insertions, deletions, recombination, or shuffling. Specialized codon-aware Operations are provided for transformations that operate on open

reading frames. *Composition Operations* combine sequences from multiple parent Pools, either by concatenating parent sequences end-to-end (the *join* Operation) or by merging multiple Pools into one Pool (the *stack* Operation). *State Operations* affect which sequences are in a Pool and how they are ordered, and include methods to select, reorder, filter, and replicate sequences.

Sequence generation

To generate a sequence, PoolParty passes a *state*—an integer index that, together with a random seed, uniquely determines the output—to the root Pool. Generation then proceeds in two passes (Fig. 1B). In the *backward pass*, PoolParty propagates the state from the root Pool back through the DAG. Each Operation decomposes the state it receives into an internal state and one or more states that are then passed to its input Pools. Each of the input Pools then passes its assigned state to its upstream Operation. In the *forward pass*, each Operation generates a sequence based on its input sequences (if there are any) and its internal state. This process starts at the source Operations at the leaves of the DAG and proceeds forward until the root Pool receives the completed sequence. To generate a full library, PoolParty iterates the state passed to the root Pool from zero up to the desired library size, recording the individual sequences as they are generated.

Operation modes

Each Operation has a mode that determines how it interprets its internal state. Operations can run in one of three modes:

- In **sequential mode**, the Operation exhaustively enumerates a finite set of design choices. For example, when *mutagenize* is instructed to generate single-nucleotide mutations in sequential mode, it yields one variant for each possible character change at each target position. The internal state indexes which specific mutation is applied.
- In **random mode**, design choices are sampled stochastically. For example, the *shuffle* Operation always operates in random mode, randomly reordering the bases in a sequence. Users can fix the number of distinct random designs used (and thus apply the same random operation to multiple sequences) or instruct PoolParty to apply a different random design every time the Operation is triggered. A master seed ensures that this process is reproducible.
- In **fixed mode**, the output is uniquely determined by the input sequences. For example, *join* concatenates its input sequences without modification, and therefore has no use for an internal state.

StateTracker

Each Operation’s backward-pass decomposition is designed to reverse its forward-pass composition. For most Operations, the output state is the Cartesian product of the Operation’s internal state and the states of the input Pools. In such cases, the Operation uses mixed-radix decomposition to split the state into its component states. The *stack* Operation is different: its output states are the disjoint union of its input Pool states. The *stack* Operation therefore uses the state it is fed to select which input Pool it will use, and to instruct this Pool (but not the other Pools) which sequence to return. Keeping track of these rules across a complex DAG requires specialized bookkeeping. We

therefore developed a back-end package called StateTracker that automates state composition and decomposition during forward and backward passes through the DAG.

Sequence regions

One often wants to perform different operations on different parts of a template sequence. To facilitate this, PoolParty lets users mark regions of interest within sequences using XML-style tags. PoolParty supports both opening/closing tag pairs (e.g., `AAA<cre>CCCCGGG</cre>TTT`), as well as self-closing tags (e.g., `ACGT<ins/>ACGT`). Operations can then be instructed to act only on a specified region, as illustrated in Fig. 1C. Tags persist through the DAG and remain valid even when upstream Operations change the content of a region. Multiple Operations can thus target modifications to the same region in series.

Sequence metadata

It is often useful to record how each sequence within a library was constructed. PoolParty provides three complementary ways to do this (Fig. 1C).

- **Design cards** record the specific changes applied to each sequence by each operation. Users specify which Operations and Operation-specific variables to track. The resulting cards are returned as a DataFrame: each row corresponds to a sequence, with columns recording the values of the operation-specific variables. These methods can thus support downstream filtering, grouping, and modeling without the need to parse generated sequences.
- **Sequence names** summarize at a glance how each sequence was constructed. Users assign a prefix to each Operation they wish to track. PoolParty then combines each prefix with the corresponding Operation's internal state, and concatenates these to form the sequence's name. For example, `wt.mut_03.bc_01` denotes a wild-type sequence with mutation variant number 3 and barcode number 1.
- **Sequence styling** allows sequences to be annotated with colors and formatted text that reflects each one's construction history. For example, `mutagenize` can be instructed to mark the positions it mutates in bold yellow text, making these positions stand out from the rest of the sequence. This formatting is accomplished using ANSI escape codes, so output renders correctly in terminals, Jupyter notebooks, and other environments that support text formatting standards. Importantly, styles combine in a natural manner as sequences propagate through the DAG, e.g., a mutated position can be underlined even if it occurs within a sequence region that is already colored blue.

Implementation and extensibility

PoolParty requires Python 3.10 or later, is designed to have minimal dependencies, and is distributed via PyPI. Documentation, tutorials, and example notebooks are available on the ReadTheDocs and GitHub project websites. PoolParty is designed to be extensible: users can create new Operation types by subclassing the Operation base class and implementing a few required methods. Custom Operations automatically inherit support for state tracking, design card generation, and visualization, allowing researchers to add domain-specific functionality without reimplementing core infrastructure.

SpliceAI surrogate analysis

This section provides technical details for the surrogate modeling analysis presented in Results. We sampled 9-mers from the human 5' splice-site (5'ss) position weight matrix [Wong et al., 2018] and scored them with MaxEntScan [Yeo and Burge, 2004]. Sequences were binned into 100 MaxEntScan score (hereafter, strength) quantiles, and 20 sequences were drawn from each bin (2,000 total). Each 9-mer was inserted at 100 positions flanking the SMN2 exon 7 canonical donor and paired with a matched control in which the GT dinucleotide was disrupted (T>A at +2; Fig. 4A).

We predicted SpliceAI donor probabilities at the canonical site using the five-model ensemble of Jaganathan et al. [2019] with ± 5 kb of genomic context (GRCh38). For each library-control pair, we computed the difference in logit-transformed predictions at the canonical donor. These per-sequence values were averaged within each cell of the resulting 100×100 position $\times$ strength grid (Fig. 4B).

We fit generalized additive models (GAMs) to the cell means using pyGAM [Servén and Brummitt, 2018], with smooth terms for position, strength, and their interaction. Models were weighted by the inverse variance of each cell mean. Exonic and intronic regions were modeled separately. Model performance was evaluated by checkerboard cross-validation of the grid.

Example Applications

DMS library for protein GB1

As a first example, we use PoolParty to reconstruct and extend the library used in a DMS experiment on protein GB1 by Olson et al. [2014]. Their library contained the wild-type GB1 coding sequence, all single amino acid mutations (excluding the start codon), and nearly all possible pairs of amino acid mutations. This library is substantially larger than most DMS libraries, containing over half a million variants. Its inclusion of so many pairwise variants has helped it become a benchmark dataset in the quantitative modeling of protein sequence-function relationships [Otwinowski, 2018, Otwinowski et al., 2018, Tareen et al., 2022, Faure and Lehner, 2024].

Fig. 2A illustrates the DAG used to construct this library; the code that establishes the DAG is shown in Fig. 2B. First we define a Pool, named `orf_pool`, that represents the wild-type GB1 coding sequence. This Pool is then fed as input to four separate branches. The first two branches use the `mutagenize_orf` Operation in sequential mode to enumerate all 1,045 single and all 546,085 pairwise amino acid substitutions. A third branch uses `mutagenize_orf` in random mode to generate 10,000 variants with mutations at 10% of amino acid positions. Because `mutagenize_orf` operates at the codon level, users specify

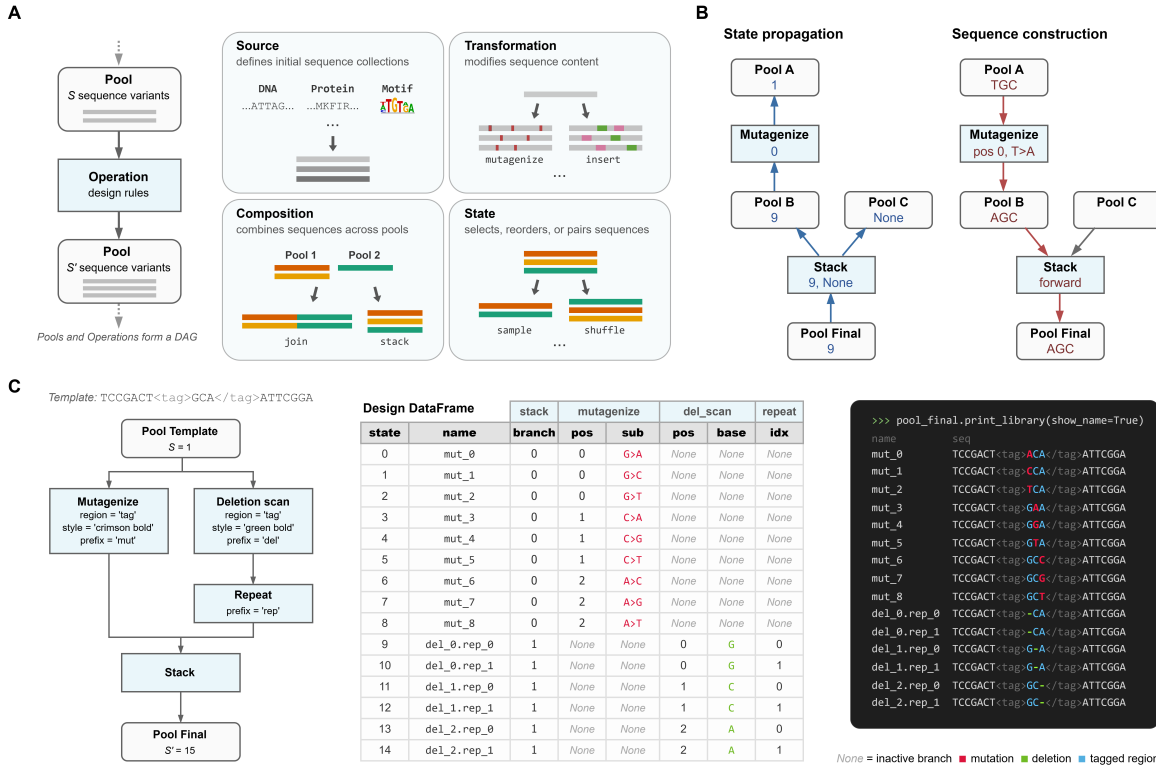


Fig. 1: PoolParty builds libraries from Pools and Operations. (A) Pools represent collections of sequences and Operations generate new Pools based on their inputs (left). Users chain Operations into a directed acyclic graph (DAG) that specifies the desired library, but delays sequence generation until the user requests it. Operations fall into four functional categories: source, transformation, composition, and state (right). (B) Sequence generation proceeds in two passes. For each desired sequence, PoolParty passes a number (called the *state*) that specifies which sequence to generate to the root Pool. This state is propagated backward through the DAG, resolving the internal state of every Operation along the way (left). A subsequent forward pass then causes each Operation to generate and combine sequence fragments according to its resolved state, producing the final sequence at the root Pool (right). When multiple Pools are combined by **stack**, only one parent is active for a given state; inactive branches receive no state (*None*) and are shown as grey arrows. (C) Example workflow. The template sequence contains a user-specified region (called “tag”) that is targeted for both mutagenesis and deletion scanning. Left: the DAG specifying the sequence library. Center: the design card DataFrame returned by the DAG. Right: styled sequence output, with colors indicating mutations (red), deletions (green), and the tagged region (blue).

only the desired amino acid substitutions, and PoolParty resolves the corresponding nucleotide changes. A fourth branch uses the **repeat** Operation to produce 100 wild-type replicates. All four branches are then merged by **stack** into a final library comprising 557,230 sequences. The design card provided with each variant (Fig. 2C) reports the component it derives from, the mutations it contains, and the replicate index (for wild-type sequences). Users can inspect the structure of the DAG by using the `print_dag` method (Fig. 2D).

MPRA library design for regulatory grammar

MPRAs use synthetic DNA sequences to dissect the cis-regulatory codes underlying a variety of gene regulatory steps. Unlike DMS experiments, MPRAs for regulatory grammar often involve systematically changing the spacing, order, orientation, and multiplicity of known regulatory motifs within an otherwise inert sequence. The combinatorial variation of these design parameters reveals how regulatory proteins bound at cognate sites orchestrate regulatory activity. For example, Georgakopoulos-Soares et al.

[2023] tested over 200,000 sequences with all pairwise and triplet arrangements of 18 TFBSs in synthetic enhancers, and found that orientation and order are major drivers of expression. The scale and controlled design of these data also make them highly suited for training and benchmarking machine learning models [Fleur et al., 2024].

As a second use case, we designed an MPRA library studying how binding site arrangements of three liver-enriched TFs affect regulatory activity (Fig. 3A,B). Following the oligo layout of Melnikov et al. [2012], we define the template Pool from a sequence containing an inert 100 bp CRE region and a barcode region, both tagged with region markers (Section 2.3). Each of the three TFs — HNF4A, PPARA, and XBP1 — is represented by a Pool containing both the forward and reverse-complement binding sites. We use `replacement_multiscan` in sequential mode to tile these Pools within the central 80 bp of the CRE without overlap. PoolParty automatically enumerates 2,184 valid placements, each in eight orientation combinations, for a CRE Pool of 17,472 variants. After repeating this Pool three times, we fill the barcode

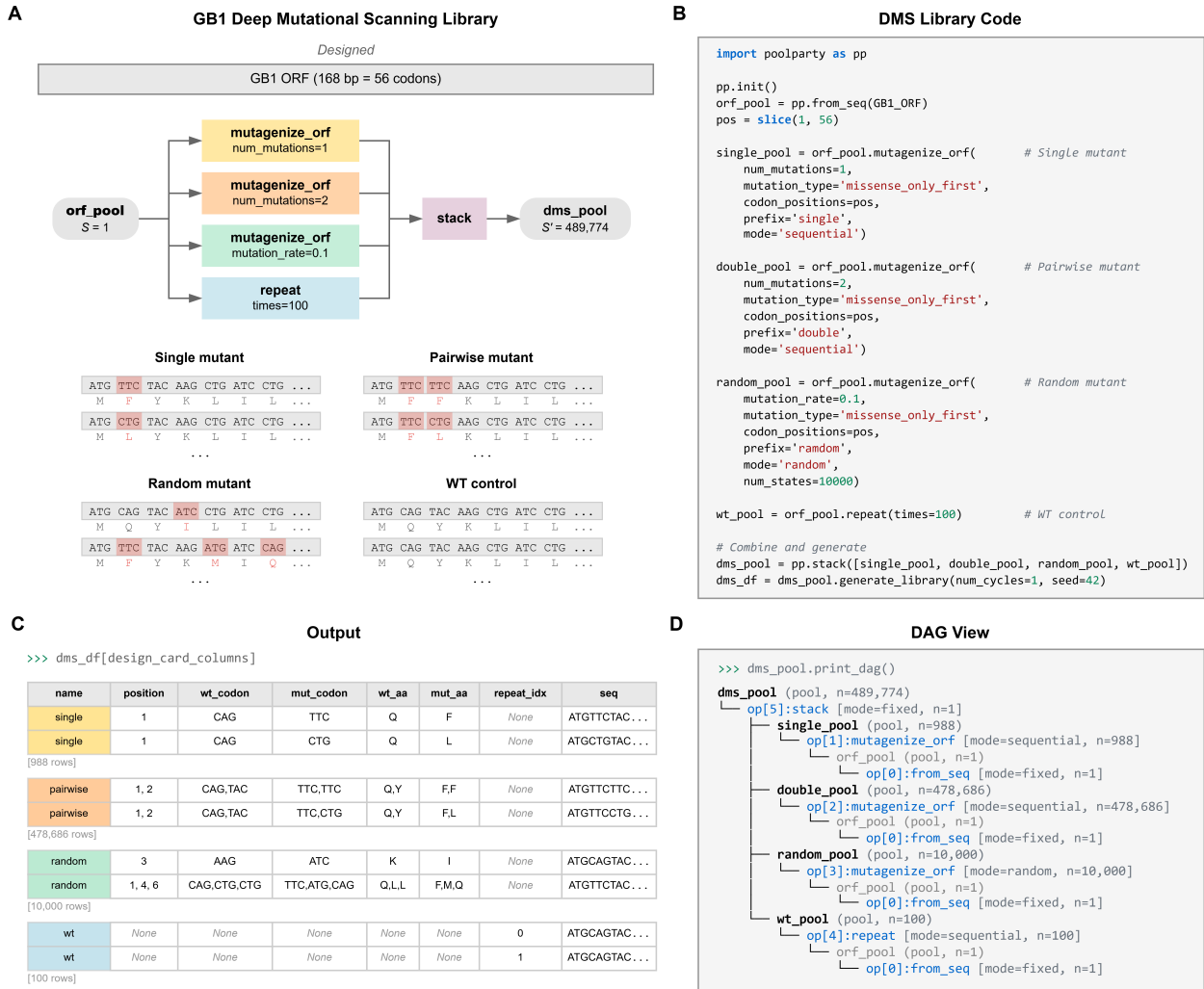


Fig. 2: PoolParty design of a GB1 deep mutational scanning library. (A) DAG and representative sequences for each of the four library components: single and pairwise amino acid substitutions, random higher-order mutants, and wild-type replicates. Mutated codons are highlighted with amino acid translations shown beneath. (B) Python code implementing the DAG. (C) Design cards reporting the library component, mutation positions, and amino acid changes for representative variants. (D) DAG structure displayed by `print_dag()`.

region with random barcodes from `get_barcodes`, giving the final `mpira_pool` of 87,360 sequences at a 1:3 CRE-to-barcode ratio (Fig. 3A).

PoolParty supports two types of styling. The style parameter of Operations marks the modifications they introduce to the sequence, while the `stylize` Operation annotates a region of the existing sequence. We first use `stylize` to add a background color to the tagged CRE region in the template. For the inserts, we color each TFBS Pool using the style parameter in `from_seqs` (HNF4A blue, PPARA purple, XBP1 orange) and mark barcodes bold in `get_barcodes` (Fig. 3B). When the TFBSs are tiled into the CRE, their colors overlay the CRE region background color, and both are rendered in the `print_library` output (Fig. 3C), making the positions and ordering of the three TFBSs immediately visible.

Surrogate modeling of SpliceAI predictions

In silico libraries designed in the same spirit as MPRA are increasingly used to probe the regulatory mechanisms learned by genomic deep learning models, treating these models as virtual assays [Avsec et al., 2021, Toneyan and Koo, 2024, Seitz et al., 2024, Nagai et al., 2026]. After scoring these libraries with the model, the relationships between perturbed sequence features and predicted outcomes can be summarized by functions of simpler mathematical form that are intrinsically interpretable, an approach called surrogate modeling. PoolParty streamlines this process through design cards, which record the sequence changes defining each variant and can serve directly as covariates for surrogate models. As a third use case, we apply this approach to characterize the effect of cryptic donor insertion on canonical 5'ss strength as judged by SpliceAI [Jaganathan et al., 2019].

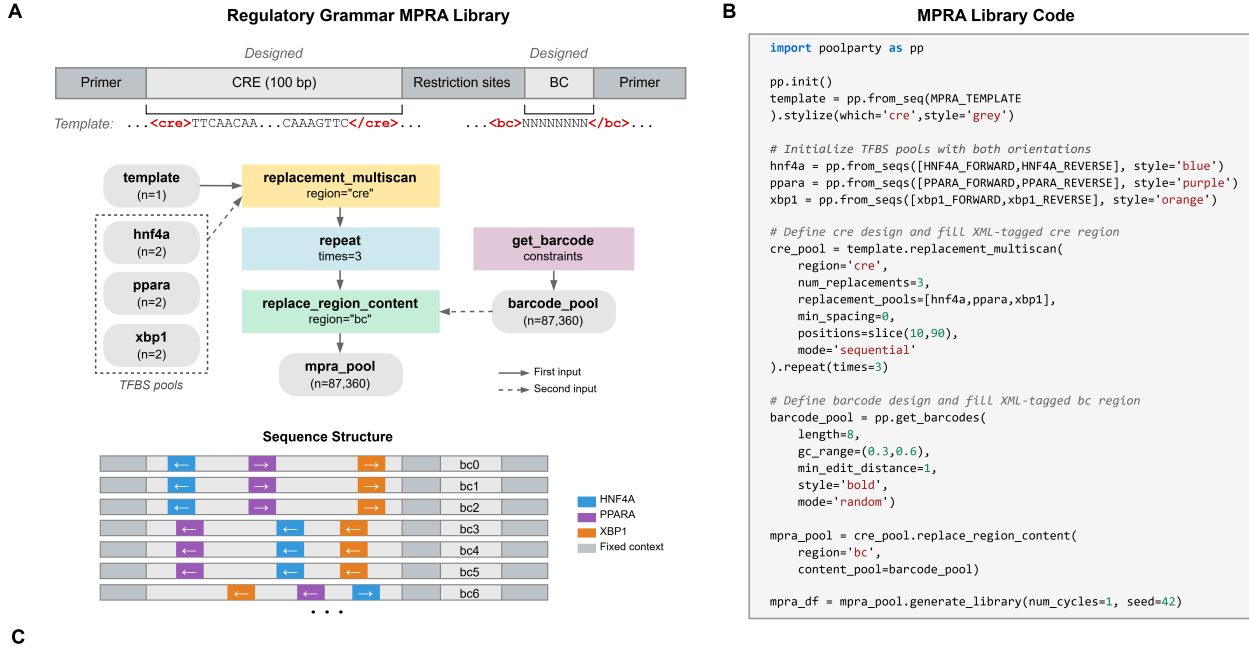


Fig. 3: Design of a regulatory grammar MPRA library using PoolParty. (A) Schematic of the regulatory grammar MPRA library targeting a 100-bp CRE. HNF4A, PPARG, and XBP1 binding sites are tiled across the CRE in different orders and orientations, with each variant assigned three unique barcodes (87,360 sequences total). Representative sequence structures are shown, with TFBS placements and barcode assignments indicated. (B) Python code for library construction. TFBS pools are initialized with both forward and reverse-complement sequences to represent orientation. (C) Example design cards. Representative rows are shown for each CRE variant, with insertion position, orientation, and barcode index recorded.

Using PoolParty, we designed a library by varying the strength (MaxEntScan) of inserted cryptic donors and their position relative to the canonical 5'ss of SMN2 exon 7 (Methods). The library covered a full grid of 100 positions and 100 strength quantile bins, with design cards automatically recording these parameters for each variant (Fig. 4A). We paired each variant with a matched control and defined the cryptic donor effect as the difference in SpliceAI predictions at the canonical 5'ss between the two. Separate generalized additive models (GAMs) were fit to this quantity for exonic and intronic regions, using only smooth spline terms for strength, position, and their interaction (Fig. 4A).

The GAMs achieved a held-out R^2 of 0.82 (exon) and 0.71 (intron), and their response surfaces matched the observed data (Fig. 4B,C). Positional effects showed stronger suppression for exonic than intronic cryptic sites (Fig. 4D), with effects peaking at 20 bp upstream, consistent with a prior report of preferential depletion of decoy donors proximal to annotated 5'ss on the exonic side [Dawes et al., 2022]. In terms of strength effects, stronger cryptic donors drove larger suppression in both models (Fig. 4E). Positional and strength effects also interacted in a non-additive manner, as shown by the interaction surface (Fig. 4F). We compared the GAMs to simpler surrogate alternatives and found that incorporating non-linearity and interaction terms enhanced predictive performance (Fig. 4G).

Discussion

We have presented PoolParty, a Python package that replaces the ad hoc scripting commonly used to design oligonucleotide libraries with a declarative, composable, and flexible framework. We demonstrated PoolParty by considering three use cases: library design for a DMS experiment on protein GB1; library design for an MPRA studying transcriptional regulatory grammar; and a surrogate modeling analysis of the effects that cryptic donor sites have on the alternative splicing predictions of SpliceAI. These examples illustrate the flexibility of PoolParty and its applicability to experimental and computational studies.

We wish to highlight the design card that PoolParty returns with each generated sequence. By providing a structured record of how each sequence was constructed, PoolParty facilitates the use of sequence design parameters as covariates in downstream analysis. In the SpliceAI example, the design cards enabled surrogate modeling without any post hoc sequence parsing; we expect this pattern to generalize to the systematic interrogation of other genomic AI models.

PoolParty was designed to address a focused computational problem: provide a lightweight tool for streamlining the design of DNA sequence libraries. Although PoolParty does provide a `filter` Operation that allows users to remove undesirable sequences from a Pool, it is not intended as a bona fide sequence optimization tool, e.g., for designing synthetic proteins or cis-regulatory elements. Moreover, the current implementation generates sequences one at a time, requiring roughly one

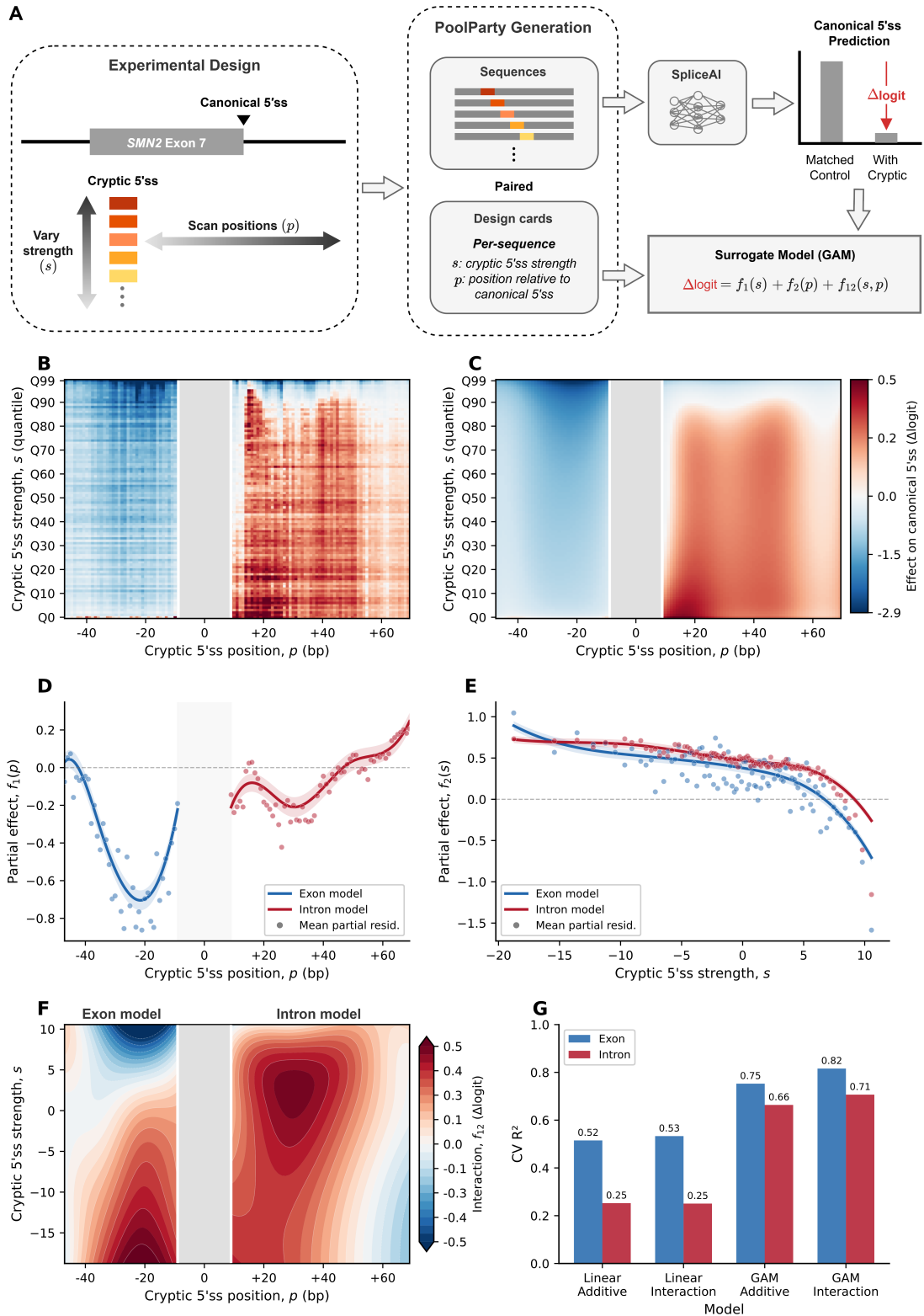


Fig. 4: Surrogate modeling of SpliceAI predictions using design-card features. (A) Experimental design. Cryptic 5'ss 9-mers spanning a range of MaxEntScan strengths (s) are inserted at positions (p) flanking the SMN2 exon 7 canonical donor. Paired library (GT) and control (GA) sequences are generated by PoolParty, and s and p are recorded in design cards. SpliceAI predictions at the canonical donor are summarized as Δlogit and modeled as a function of s and p using a generalized additive model (GAM). (B) Mean Δlogit across the strength $\times$ position grid (100×100 ; each cell averages 20 distinct 9-mers). The grey band marks the canonical donor region. (C) GAM interaction model predictions using the same color scale as in B. (D) Partial dependence of the position term $f_1(p)$, shown separately for exon (blue) and intron (red) models. Shaded regions indicate 95% confidence intervals; points denote mean partial residuals. (E) Partial dependence of the strength term $f_2(s)$, with conventions as in D. (F) Interaction surface $f_{12}(s, p)$ for exon and intron models. (G) Block cross-validation R^2 for linear and GAM models, with and without interaction terms.

millisecond per sequence on a standard laptop. This suffices for typical oligonucleotide libraries but could bottleneck large-scale tasks; a future implementation of batch-mode sequence generation would address this.

We therefore anticipate that PoolParty will substantially ease the design of MPRA and DMS experiments, as well as other high-throughput functional genomics assays. And as genomic AI models become increasingly accurate, PoolParty will accelerate efforts to gain insight into the molecular mechanisms that underlie the predictions of these models.

Competing interests

The authors declare no competing interests.

Author contributions statement

Z.L. conceived the project. Z.L. and J.B.K. designed the project, wrote the software, performed the analyses, and wrote the manuscript. J.B.K. supervised the research.

Acknowledgments

This work was supported in part by: NIH grants R01HG011787 (J.B.K., Z.L.). Computational equipment was supported by NIH grant S10OD028632.

References

- Žiga Avsec, Melanie Weilert, Avanti Shrikumar, Sabrina Krueger, Amr Alexandari, Khyati Dalal, Robin Fropf, Charles McAnany, Julien Gagneur, Anshul Kundaje, and Julia Zeitlinger. Base-resolution models of transcription-factor binding reveal soft motif syntax. *Nature Genetics*, 53(3):354–366, March 2021. ISSN 1546-1718. doi: 10.1038/s41588-021-00782-6.
- Luca Barbon, Victoria Offord, Elizabeth J Radford, Adam P Butler, Sebastian S Gerety, David J Adams, Hong Kee Tan, and Andrew J Waters. Variant library annotation tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. *Bioinformatics*, 38(4):892–899, 2021. ISSN 1367-4803. doi: 10.1093/bioinformatics/btab776.
- Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cyron J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009. ISSN 1367-4803. doi: 10.1093/bioinformatics/btp163.
- Ruebena Dawes, Himanshu Joshi, and Sandra T. Cooper. Empirical prediction of variant-activated cryptic splice donors using population-based RNA-Seq data. *Nature Communications*, 13(1):1655, March 2022. ISSN 2041-1723. doi: 10.1038/s41467-022-29271-y.
- Andre J. Faure and Ben Lehner. MoCHI: Neural networks to fit interpretable models and quantify energies, energetic couplings, epistasis, and allostery from deep mutational scanning data. *Genome Biology*, 25(1):303, December 2024. ISSN 1474-760X. doi: 10.1186/s13059-024-03444-y.
- Alyssa La Fleur, Yongsheng Shi, and Georg Seelig. Decoding biology with massively parallel reporter assays and machine learning. *Genes & Development*, 38(17-20):843–865, September 2024. ISSN 0890-9369, 1549-5477. doi: 10.1101/gad.351800.124.
- Douglas M Fowler and Stanley Fields. Deep mutational scanning: a new style of protein science. *Nat Methods*, 11(8):801 – 807, 2014. ISSN 1548-7105. doi: 10.1038/nmeth.3027.
- Ilias Georgakopoulos-Soares, Naman Jain, Jesse M Gray, and Martin Hemberg. MPRAAnator: a web-based tool for the design of massively parallel reporter assay experiments. *Bioinformatics*, 33(1):137–138, 2016. ISSN 1367-4803. doi: 10.1093/bioinformatics/btw584.
- Ilias Georgakopoulos-Soares, Chengyu Deng, Vikram Agarwal, Candace S. Y. Chan, Jingjing Zhao, Fumitaka Inoue, and Nadav Ahituv. Transcription factor binding site orientation and order are major drivers of gene regulatory activity. *Nature Communications*, 14(1):2333, April 2023. ISSN 2041-1723. doi: 10.1038/s41467-023-37960-5.
- Andrew R Ghazi, Edward S Chen, David M Henke, Namrata Madan, Leonard C Edelstein, and Chad A Shaw. Design tools for MPRA experiments. *Bioinformatics*, 34(15):2682–2683, 2018. ISSN 1367-4803. doi: 10.1093/bioinformatics/bty150.
- Fumitaka Inoue and Nadav Ahituv. Decoding enhancers using massively parallel reporter assays. *Genomics*, 106(3):159 – 164, 2015. ISSN 0888-7543. doi: 10.1016/j.ygeno.2015.06.005.
- Kishore Jaganathan, Sofia Kyriazopoulou Panagiotopoulou, Jeremy F. McRae, Siavash Fazel Darbandi, David Knowles, Yang I. Li, Jack A. Kosmicki, Juan Arbelaez, Wenwu Cui, Grace B. Schwartz, Eric D. Chow, Efstathios Kanterakis, Hong Gao, Amirali Kia, Serafim Batzoglou, Stephan J. Sanders, and Kyle Kai-How Farh. Predicting Splicing from Primary Sequence with Deep Learning. *Cell*, 176(3):535–548.e24, January 2019. ISSN 0092-8674, 1097-4172. doi: 10.1016/j.cell.2018.12.015.
- Justin B Kinney and David M McCandlish. Massively parallel assays and quantitative sequence-function relationships. *Annual Review of Genomics and Human Genetics*, 20(1): 99–127, 08 2019. ISSN 1527-8204. doi: 10.1146/annurev-genom-083118-014845. Wrote.
- Adam Klie and David Laub. SeqPro (sequence processing toolkit), 2023. URL <https://github.com/ML4GLand/SeqPro>.
- Alexandre Melnikov, Anand Murugan, Xiaolan Zhang, Tiberiu Tesileanu, Li Wang, Peter Rogov, Soheil Feizi, Andreas Gnirke, Curtis G Callan, Justin B Kinney, Manolis Kellis, Eric S Lander, and Tarjei S Mikkelsen. Systematic dissection and optimization of inducible enhancers in human cells using a massively parallel reporter assay. *Nature Biotechnology*, 30(3):271–277, 02 2012. ISSN 1087-0156. doi: 10.1038/nbt.2137. Wrote (coauthored).
- Masayuki Nagai, Alan E. Murphy, Kaeli Rizzo, and Peter K. Koo. Toward Interpretable and Generalizable AI in Regulatory Genomics, February 2026.
- C Anders Olson, Nicholas C Wu, and Ren Sun. A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Current biology : CB*, 24(22):2643 – 2651, 11 2014. ISSN 0960-9822. doi: 10.1016/j.cub.2014.09.072.
- Jakub Otwinowski. Biophysical inference of epistasis and the effects of mutations on protein stability and function. *Mol Biol Evol*, 35(10):2345 – 2354, 10 2018. ISSN 0737-4038. doi: 10.1093/molbev/msy141.
- Jakub Otwinowski, David M McCandlish, and Joshua B Plotkin. Inferring the shape of global epistasis. *Proc Natl Acad Sci USA*, 08 2018. ISSN 0027-8424. doi: 10.1073/pnas.1804015115.
- Filipa Pereira, Flávio Azevedo, Ângela Carvalho, Gabriela F Ribeiro, Mark W Budde, and Björn Johansson. Pydna: a

- simulation and documentation tool for DNA assembly strategies using python. *BMC Bioinformatics*, 16(1):142, 2015. doi: 10.1186/s12859-015-0544-x.
- Jacob Schreiber. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. *bioRxiv*, page 2025.08.08.669296, 2025. doi: 10.1101/2025.08.08.669296.
- Evan E. Seitz, David M. McCandlish, Justin B. Kinney, and Peter K. Koo. Interpreting cis-regulatory mechanisms from genomic deep neural networks using surrogate models. *Nature Machine Intelligence*, 6(6):701–713, June 2024. ISSN 2522-5839. doi: 10.1038/s42256-024-00851-5.
- Daniel Servén and Charlie Brummitt. pyGAM: Generalized additive models in python, March 2018. URL <https://doi.org/10.5281/zenodo.1208723>.
- Neil Swainston, Andrew Currin, Lucy Green, Rainer Breitling, Philip J Day, and Douglas B Kell. CodonGenie: optimised ambiguous codon design tools. *PeerJ Computer Science*, 3: e120, 2017. doi: 10.7717/peerj-cs.120.
- Ammar Tareen, Mahdi Kooshkbaghi, Anna Posfai, William T. Ireland, David M. McCandlish, and Justin B. Kinney. MAVE-NN: learning genotype-phenotype maps from multiplex assays of variant effect. *Genome Biology*, 23(1):98, 2022. ISSN 1474-7596. doi: 10.1186/s13059-022-02661-7.
- Shushan Toneyan and Peter K. Koo. Interpreting cis-regulatory interactions from large-scale deep neural networks. *Nature Genetics*, 56(11):2517–2527, November 2024. ISSN 1546-1718. doi: 10.1038/s41588-024-01923-3.
- Mandy S. Wong, Justin B. Kinney, and Adrian R. Krainer. Quantitative Activity Profile and Context Dependence of All Human 5' Splice Sites. *Molecular Cell*, 71(6):1012–1026.e3, September 2018. ISSN 1097-2765. doi: 10.1016/j.molcel.2018.07.033.
- Gene Yeo and Christopher B. Burge. Maximum Entropy Modeling of Short Sequence Motifs with Applications to RNA Splicing Signals. *Journal of Computational Biology*, 11(2-3):377–394, March 2004. ISSN 1066-5277. doi: 10.1089/1066527041410418.
- Valentin Zulkower and Susan Rosser. DNA chisel, a versatile sequence optimizer. *Bioinformatics*, 36(16):4508–4509, 2020. ISSN 1367-4803. doi: 10.1093/bioinformatics/btaa558.